Parallel Simulation of the Damped Wave Equation: A Comparative Study Using OpenMP, MPI, and CUDA

Allione Paolo Frulla Edoardo Porro Michele
345215 343236 342079
Quantum Engineering – High Performance Computing Course
Academic Year 2025–26

Abstract

This report presents the design, implementation, and performance evaluation of a numerical simulator for the two-dimensional damped wave equation, developed progressively using three different parallel programming paradigms: OpenMP, MPI, and CUDA. The first part of the project implements a shared-memory parallel solver based on a finite-difference discretization of the governing partial differential equation, producing a sequence of grayscale frames later assembled into a video. The second part extends the simulator to execute multiple independent wave simulations concurrently using a hybrid MPI+OpenMP approach, modeling single-source and multi-source (interfering) wave phenomena. The third part introduces GPU-accelerated post-processing with CUDA, where each simulation frame is recolored to emphasize wave maxima, minima, and propagation fronts. For each part, the numerical method, the parallelization strategy, and the experimental results – including correctness validation and performance scaling – are discussed.

1 Introduction

The damped wave equation is a second-order partial differential equation that models the propagation of a perturbation through a medium subject to energy dissipation, such as friction, viscosity, or internal resistance. In two spatial dimensions, the equation is written as

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = c^2 \nabla^2 u, \tag{1}$$

where $u(x, y, t)$ represents the wave height on the plane (x, y) at time t, γ is the damping coefficient, and c is the propagation speed of the wave. Compared to the ideal, non-dissipative wave equation, the term $\gamma\, \partial u/\partial t$ progressively attenuates the amplitude of the wave as it propagates.

Solving Equation (1) numerically requires discretizing both the spatial domain (grid spacing $\Delta x = \Delta y$) and the temporal domain (time step Δt), replacing the continuous function $u(x, y, t)$ with a discrete field $u^n_{i,j}$, where (i, j) are the grid indices and n is the discrete time step. Using a second-order central difference approximation for both time derivatives and a five-point stencil for the Laplacian operator, the update rule for the wave field at time step $n + 1$ is obtained as

$$\nabla^2 u^n_{i,j} \approx \frac{u^n_{i+1,j} + u^n_{i-1,j} + u^n_{i,j+1} + u^n_{i,j-1} - 4u^n_{i,j}}{\Delta x^2}, \tag{2}$$

$$u^{n+1}_{i,j} = \frac{2u^n_{i,j} - \left(1 - \dfrac{\gamma \Delta t}{2}\right) u^{n-1}_{i,j} + c^2 \Delta t^2 \nabla^2 u^n_{i,j}}{1 + \dfrac{\gamma \Delta t}{2}}. \tag{3}$$

Equation (3) forms the computational core of all three parts of this project, and it is evaluated, frame by frame, for every grid point of an $M \times M$ matrix, over N simulation steps. This report is organized into three sections, each corresponding to one part of the assignment: Section 2 describes the shared-memory OpenMP implementation of the base simulator; Section 3 presents the MPI-based extension that executes three concurrent simulations, including single- and multi-source wave interference; and Section 4 discusses the CUDA-accelerated frame colorization stage. Each section introduces the corresponding sub-problem and is followed by a placeholder for the experimental results obtained on the target HPC cluster.

2 Part 1 – Shared-Memory Simulation with OpenMP

2.1 Problem Description

The first part of the project requires the implementation of a sequential-to-parallel simulator of the damped wave equation introduced in Section 1 (Eq. 1), parallelized using OpenMP. Given the physical parameters γ (damping) and c (propagation speed), and the numerical parameters Δt, Δx, the grid size M, and the number of time steps N, the simulator must compute the evolution of the wave height matrix $u^n_{i,j}$ starting from an initial condition where the matrix is uniformly zero except for an initial impulse applied at a boundary point (i_0, j_0) at $n = 0$.

2.2 Numerical Method and Implementation

At each time step, the simulator applies the explicit update rule of Equation (3) using a nine-point isotropic Laplacian stencil, which requires keeping in memory the wave fields at the two previous time steps (u^n and u^{n-1}) to compute u^{n+1}. The computation is organized as a loop over the $N - 1$ frames to be generated (the initial condition is written as frame 0 before the loop starts); for each

frame, the update is applied independently to every interior grid point (i, j), which makes the spatial loop embarrassingly parallel and well suited to a `#pragma omp parallel for` directive. In the current implementation the directive is applied with `schedule(static)` to the outer i loop only (a row-wise domain decomposition), rather than collapsing the nested i, j loops: since the per-point workload is uniform, a static row-wise split is sufficient to balance the work evenly across threads without the extra overhead of collapsing the two loops. The same parallelization pattern (a `#pragma omp parallel for schedule(static)` over rows, or over the flattened array, respectively) is also used to initialize the fields at $n = 0$ and to rescale each computed frame to $[0, 255]$.

Boundary points are excluded from the update loop (the indices i, j range over $[1, M - 2]$) and are therefore left at their initial value rather than being explicitly reflected or clamped; since the initial Gaussian pulse is confined to a region around (i_0, j_0) well inside the domain, the border cells effectively behave as a fixed (Dirichlet-like) boundary equal to zero, provided M is large enough that the wavefront does not reach the edges during the simulated interval.

After each time step is computed, the resulting matrix is rescaled to the $[0, 255]$ range and written to disk as a grayscale image in PGM (Portable Graymap) format, following the P5 binary encoding, into a dedicated `sim` folder, with filenames of the form `frame_%05d.pgm`. The minimum and maximum values used for the rescaling are derived once, before the time loop, directly from the pulse `intensity` parameter (i.e. $[-|\text{intensity}|, +|\text{intensity}|]$), rather than being recomputed frame by frame from the actual data range; this keeps the gray-level mapping consistent across the whole video at the cost of clipping any point whose amplitude exceeds the assumed range. Once all N frames have been generated, the FFmpeg tool is used to assemble the image sequence into an MP4 video, with the frame rate chosen consistently with the simulation time step Δt (i.e., fps $= 1/\Delta t$, suitably scaled for a perceptually smooth playback).

2.3 Parallelization Strategy

The spatial update loop is parallelized with a single `#pragma omp parallel for` directive applied to the outer `i` loop, using `schedule(static)`. Since every interior grid point (i, j) is updated independently and writes only to its own entry in the `new` array — reading exclusively from the read-only `old` and `current` arrays of the previous time steps — the workload is perfectly uniform across iterations and requires no synchronization within the loop body. Static scheduling was therefore preferred over dynamic scheduling: since each grid row involves an identical amount of work, a fixed, contiguous partitioning of rows among threads avoids the runtime overhead of dynamic work assignment while still guaranteeing a balanced load. Row-major partitioning also preserves cache locality, as each thread operates on a contiguous block of rows and the memory access pattern within `wave_update_9_pts` exploits spatial locality.

No false sharing is expected between threads, since `schedule(static)` assigns each thread a contiguous chunk of full rows (each of length M), and adjacent threads' data regions are separated by an entire row rather than by individual cache-line-sized elements. The same parallelization pattern is applied independently to the color-rescaling loop (`rescale_discretize_intensity`) after each time step, which is likewise embarrassingly parallel.

The number of OpenMP threads was varied across $T \in \{1, 2, 4, 8, 16\}$ to evaluate scaling behavior, using `OMP_NUM_THREADS` together with `OMP_PROC_BIND=true` and `OMP_PLACES=cores` to pin threads to physical cores and avoid migration overhead. As discussed in Section 2.4, profiling with Intel VTune revealed that, for the tested grid sizes, the per-frame PGM file-writing routine (`write_snapshot_serial`) — which remains strictly serial in the current implementation — accounts for a substantial fraction of total wall-clock time, independently of the thread count used for the computational kernel; this I/O-bound behavior limits the achievable speedup and is discussed further in the performance evaluation.

2.4 Results

[Insert here: simulation parameters used (Table 1), example frames showing wave propagation at different time steps (Figure 1), execution time vs. number of OpenMP threads, and the resulting speedup/efficiency curves.]

Table 1: Simulation parameters – Part 1.

Parameter	Value
γ (damping)	*TBD*
c (propagation speed)	*TBD*
Δt	*TBD*
Δx	*TBD*
M (grid size)	*TBD*
N (number of frames)	*TBD*

Figure 1: Example frames of the wave simulation at different time steps. *[Placeholder – insert figure]*

Figure 2: Speedup and efficiency as a function of the number of OpenMP threads. *[Placeholder – insert figure]*

3 Part 2 – Multi-Simulation Extension with MPI

3.1 Problem Description

The second part of the project extends the OpenMP simulator of Section 2 to the distributed-memory setting using MPI, with the goal of executing *three independent simulations simultaneously* on an HPC cluster. Each simulation continues to exploit intra-node, shared-memory parallelism through OpenMP, resulting in a hybrid MPI+OpenMP implementation. The three simulations differ in their initial conditions:

(i) **Simulation 1**: identical to the base case of Part 1, with a single impulse source at (i_0, j_0) and $t = 0$.

(ii) **Simulation 2**: two independent wave sources, located at two distinct points of the plane, sharing the same damping γ and propagation speed c, both activated at $t = 0$. This configuration is designed to highlight wave interference patterns arising from the superposition of the two wavefronts.

(iii) **Simulation 3**: identical to Simulation 2, but the second source is activated at a later time $t = t_{\text{start}} > 0$, producing an asymmetric interference pattern.

3.2 Parallelization Strategy

The three simulations are mapped onto separate MPI processes (or groups of processes), allowing them to run concurrently and independently, since there is no data dependency between them. Each MPI process executes its own instance of the OpenMP-parallelized time-stepping loop described in Section 2, writing its output frames to a dedicated folder (`sim1`, `sim2`, `sim3` respectively). This two-level parallelization scheme – MPI across simulations and OpenMP within each simulation – allows efficient use of a multi-node cluster where each node hosts one MPI process exploiting all of its available cores via OpenMP threads.

3.3 Results

[Insert here: simulation parameters for the three runs (Table 2), example frames illustrating single-source propagation and the interference patterns of Simulations 2 and 3 (Figure 3), and the measured execution time/scalability of the hybrid MPI+OpenMP implementation as a function of the number of MPI processes and OpenMP threads.]

Table 2: Simulation parameters – Part 2.

Parameter	Sim. 1	Sim. 2	Sim. 3
γ	*TBD*	*TBD*	*TBD*
c	*TBD*	*TBD*	*TBD*
Source(s) position	*TBD*	*TBD*	*TBD*
t_{start} (2nd source)	–	0	*TBD*

Figure 3: Comparison of single-source propagation and multi-source interference patterns. *[Placeholder – insert figure]*

4 Part 3 – GPU-Accelerated Frame Colorization with CUDA

4.1 Problem Description

The third part of the project focuses on improving the visual quality of the simulation output by transforming each grayscale frame into a colored RGB frame, with the goal of clearly highlighting the maxima and minima of the wave field while smoothly fading the intermediate ("calm") regions, which are colored with a neutral, bright color (e.g., white) to make the propagating wavefronts stand out. The colorization step is implemented on the GPU using CUDA, since the mapping from a scalar grayscale value to an RGB triplet is an embarrassingly parallel, per-pixel operation,

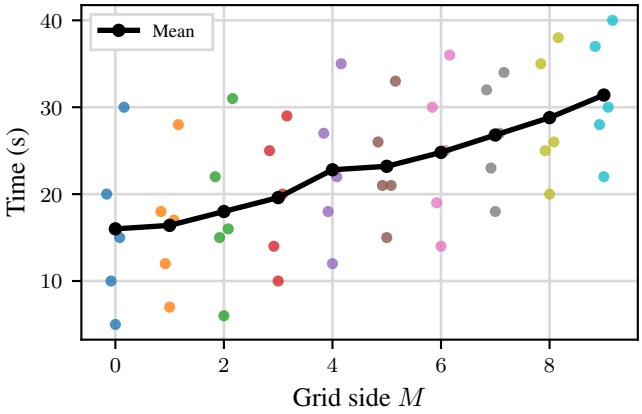

Figure 4: Execution time and scalability of the hybrid MPI+OpenMP implementation. *[Placeholder – insert figure]*

making it an ideal candidate for massive data-parallel execution.

Each colored frame is saved in PPM (Portable Pixmap) format, using the ASCII P3 encoding, in which every pixel is represented as a triplet of values $(R, G, B) \in [0, 255]^3$. The original grayscale matrix, computed by the OpenMP/MPI simulation core of Parts 1–2, is left unmodified and is used to continue the time-stepping of the simulation; only a colored copy of the frame is produced for visualization purposes.

4.2 Color Mapping and CUDA Kernel

A custom transfer function $f : [0, 255] \to [0, 255]^3$ is defined to map a grayscale intensity (representing the wave amplitude, after normalization) to an RGB color. The mapping is designed so that:

- values near the neutral/zero level of the wave are mapped to a light, low-saturation color (white or near-white), so that calm regions of the plane remain visually unobtrusive;
- values approaching the maximum amplitude are mapped to a bright, highly saturated color (e.g., red), and values approaching the minimum amplitude to a different bright color (e.g., blue), so that crests and troughs of the wave are immediately distinguishable;
- intermediate values (the rising and falling fronts of the wave) are mapped through a smooth color gradient interpolating between the neutral color and the extremal colors, in order to avoid visual banding artifacts.

This transfer function is implemented as a CUDA kernel that is launched with one thread per pixel of the $M \times M$ grayscale matrix; each thread independently reads the corresponding grayscale value, applies the color transfer function, and writes the resulting RGB triplet to the output buffer, which is then transferred back to host memory and serialized to the PPM file. Because each pixel is processed independently, the kernel requires no inter-thread synchronization or shared memory beyond what is used for performance optimization (e.g., coalesced global memory access, or optional use of constant/texture memory to store lookup tables for the color gradient).

4.3 Results

[Insert here: example colorized frames showing the maxima/minima highlighting and front fading effect (Figure 5), comparison between CPU and GPU colorization time per frame, and overall GPU kernel performance/throughput analysis (Figure 6).]

Figure 5: Colorized simulation frames highlighting wave maxima, minima, and fading fronts. *[Placeholder – insert figure]*

Figure 6: CUDA kernel execution time vs. grid size, and comparison against a CPU-only colorization baseline. *[Placeholder – insert figure]*

5 Conclusions

This report described the incremental development of a damped wave equation simulator across three parallel computing paradigms. The OpenMP implementation established a correct and efficient shared-memory baseline; the MPI extension enabled the concurrent execution of multiple independent simulations, including the study of wave interference phenomena, in a hybrid MPI+OpenMP fashion suitable for multi-node HPC clusters; and the CUDA stage demonstrated how GPU acceleration can be effectively applied to a massively data-parallel post-processing task, improving the visual interpretability of the simulation results without affecting the underlying numerical computation. *[Insert here a brief summary of the quantitative findings: best speedups obtained, scalability limits observed, and overall lessons learned.]*